

Algorithms, models, and datasets

Purpose of this document

This document explains algorithmic and data choices in Adaptive RL Gym Coach. It is written for review-panel questions on machine learning, deep learning, reinforcement learning, feature engineering, datasets, evaluation, and limitations.

System intelligence overview

Project combines four intelligence layers:

Layer	Method	Role
Perception	MediaPipe Pose pretrained deep learning model	Convert video frames into body landmarks
Feature engineering	Geometry and biomechanics rules	Convert landmarks into knee angle, hip angle, torso lean, tracking, depth, tempo, smoothness
Decision learning	DQN policy trained in simulated Gymnasium environment	Choose coaching action from state vector
Safety and explainability	Rule-based safety policy and threshold trace	Prevent unsafe actions and explain every recommendation

This architecture is intentionally hybrid. Deep learning handles vision. Engineering features make biomechanics interpretable. Reinforcement learning handles sequential coaching. Rules keep final advice safe and explainable.

Pose estimation model

Model used

MediaPipe Pose

Model role

MediaPipe Pose estimates body landmarks from each video frame. This project uses these landmarks:

left_shoulder
right_shoulder

```
left_hip
right_hip
left_knee
right_knee
left_ankle
right_ankle
```

Why used

- It is pretrained, so project does not need custom image labeling.
- It outputs normalized **x**, **y**, **z**, and **visibility** values.
- It is accurate enough for squat MVP landmark extraction.
- It runs locally, which avoids paid APIs and privacy issues.

Deep learning concept answer

MediaPipe Pose is deep learning inference. Input is image frame. Output is body keypoints. Project uses transfer learning style workflow: pretrained model extracts representations, downstream custom system makes exercise decisions.

Why project does not train pose model

Training pose estimation from scratch would require large labeled image datasets with body keypoint annotations. That is outside one-week MVP scope. Better engineering choice is to use pretrained pose model and focus implementation on squat biomechanics plus RL coaching.

Landmark filtering

Input

landmarks.csv rows:

```
frame,timestamp,landmark_name,x,y,z,visibility
```

Filter

Only landmarks with visibility greater than or equal to 0.3 are used.

Why used

Pose models can output uncertain keypoints when body parts are hidden or blurry. Visibility filtering reduces bad geometry calculations from low-confidence landmarks.

Biomechanics feature extraction

Per-frame geometry

Each valid frame produces:

Feature	Computation	Reason
Knee angle	Angle between hip, knee, ankle on both sides, then average	Squat depth and knee movement depend heavily on knee flexion
Hip angle	Angle between shoulder, hip, knee on both sides, then average	Hip hinge and squat depth signal
Torso lean	Angle between mid-hip and mid-shoulder	Detects excessive forward lean
Knee tracking proxy	Horizontal knee-to-ankle offset normalized by hip width	Proxy for knees moving away from foot path
Knee asymmetry	Left-right knee angle difference divided by 180	Proxy for uneven loading

Why not raw pixels

Raw video pixels are hard to explain. Landmark geometry gives interpretable features. Review panel can inspect how each decision was calculated.

Rep segmentation algorithm

Input

Ordered knee-angle series:

```
[(frame_id, knee_angle), ...]
```

Core idea

Squat has phase pattern:

`standing -> descent -> bottom -> ascent -> standing`

Knee angle decreases during descent, reaches minimum near bottom, then increases during ascent.

Thresholds

For each video:

```

low = min_angle + 0.38 * (max_angle - min_angle)
high = min_angle + 0.72 * (max_angle - min_angle)
minimum_rep_frames = max(8, fps * 0.45)

```

State machine

State	Condition	Meaning
standing	knee angle above low threshold	User is upright or near upright
bottom	knee angle drops below low threshold	User entered squat bottom phase
rep complete	knee angle rises above high threshold after enough frames	Full squat cycle detected

Why adaptive thresholds

Different users and camera angles produce different absolute angles. Thresholds are calculated from current video range, so segmentation adapts to session movement.

Per-rep feature extraction

For each detected rep, project writes `rep_features.csv`.

Fields:

Field	Meaning
<code>rep_id</code>	Repetition number
<code>start_frame</code>	First frame of rep
<code>end_frame</code>	Last frame of rep
<code>knee_angle_min</code>	Deepest knee flexion angle
<code>hip_angle_min</code>	Deepest hip angle
<code>torso_lean_max</code>	Maximum torso lean during rep
<code>tempo_sec</code>	Rep duration in seconds
<code>smoothness</code>	Knee-angle variation stability score
<code>fatigue_score</code>	Normalized fatigue estimate
<code>injury_risk</code>	Normalized injury-risk proxy
<code>mistake_label</code>	Main detected form issue
<code>knee_tracking_proxy</code>	Knee-to-ankle horizontal tracking score
<code>depth_proxy</code>	Normalized depth score
<code>evidence_frame</code>	Frame used as visual evidence

Mistake detection algorithm

Mistake labels are threshold-based and interpretable.

Priority order:

Mistake	Trigger
knee_tracking	knee_tracking_proxy > 0.35
forward_lean	torso_lean_max > baseline_torso_lean_max + 12
shallow_squat	knee_angle_min > baseline_knee_min + 18
unstable_motion	smoothness > 4.0
none	No threshold triggered

Why this order matters

Knee tracking is checked first because it directly contributes to injury-risk proxy. Forward lean and shallow squat use personalized baseline. Unstable motion catches jittery or inconsistent movement.

Fatigue score algorithm

Fatigue score is normalized between 0.0 and 1.0.

Inputs:

Signal	Meaning
Tempo drift	Later reps take longer than early reps
Range loss	Later reps lose range of motion compared with early reps
Jitter	Movement smoothness worsens

Formula:

```
fatigue_score =  
    0.45 * tempo_drift  
+ 0.35 * range_loss  
+ 0.20 * jitter
```

Then clamped to:

```
0.0 <= fatigue_score <= 1.0
```

Why these signals

Fatigue usually appears as slower reps, reduced depth, and less stable movement. This score approximates those effects without claiming clinical fatigue diagnosis.

Injury-risk score algorithm

Injury risk is normalized between 0.0 and 1.0.

Inputs:

Signal	Meaning
Torso risk	Torso lean exceeds user baseline
Knee risk	Knee tracking proxy exceeds safe offset
Asymmetry risk	Left-right knee angle differs too much

Formula:

```
injury_risk =  
    0.45 * torso_risk  
+ 0.40 * knee_risk  
+ 0.15 * asymmetry_risk
```

Then clamped to:

```
0.0 <= injury_risk <= 1.0
```

Important limitation

This is not medical injury prediction. It is a coaching risk proxy based on visible movement mechanics.

Reinforcement learning formulation

Why RL is appropriate

Exercise coaching is sequential. Feedback on rep 3 can affect rep 4. Too many corrections can overload the user. No feedback can be correct when form is good. Rest can be correct when fatigue rises.

This makes coaching a Markov Decision Process.

MDP design

State

State vector length: 9

Index	Feature	Range	Reason
0	knee_angle_min / 180	0..1	Normalized squat depth signal
1	hip_angle_min / 180	0..1	Hip movement signal
2	torso_lean_max / 90	0..1	Posture control signal
3	rep_id / 20 capped at 1	0..1	Session progress

Index	Feature	Range	Reason
4	<code>fatigue_score</code>	0..1	Fatigue state
5	<code>injury_risk</code>	0..1	Safety state
6	<code>repeated_mistakes</code> / 5 capped at 1	0..1	Recurring error context
7	<code>previous_action_id</code> / 7	0..1	Last feedback action
8	<code>skill_encoding</code>	0, 0.5, 1	Beginner, intermediate, advanced

Actions

Action ID	Action	Meaning
0	<code>verbal_cue</code>	General technique cue
1	<code>joint_highlight</code>	Highlight risky joint movement
2	<code>slow_tempo</code>	Ask user to slow down
3	<code>adjust_difficulty</code>	Reduce load or depth target
4	<code>recommend_rest</code>	Stop and recover before next set
5	<code>no_feedback</code>	No correction needed
6	<code>demonstration</code>	Show target movement pattern
7	<code>breathing_cue</code>	Cue brace and breathing

Reward

Reward combines next-rep improvement and penalties.

Positive reward:

- Next rep improves movement quality.
- Rest is recommended when fatigue is high.
- Joint highlight is selected when injury risk is high.
- No feedback is selected when no mistake exists.

Negative reward:

- Current injury risk is high.
- Current fatigue is high.
- Same mistake repeats.
- Too many corrections are given.

Reward expression from code:

```
reward =
    improvement_reward
- injury_risk * 0.45
- fatigue_score * 0.20
- repeated_mistake_penalty
- correction_penalty
```

Transition

In training environment, each step represents one rep. Environment updates fatigue, risk, repeated mistake count, previous action, and rep progress.

Episode

One training episode represents simulated squat session with fixed number of reps.

DQN model

Model used

Stable-Baselines3 DQN with MlpPolicy

Why DQN

- Action space is discrete.
- DQN learns Q-values for each coaching action.
- Model chooses action with highest expected return for current state.
- Good fit for panel explanation around RL because state, action, reward, and policy are visible.

Training parameters

Parameter	Value
learning rate	0.001
buffer size	5000
learning starts	100
batch size	32
gamma	0.92
training seeds	13, 17, 23, 29, 31
held-out eval seeds	1001 through 1030
default timesteps	50,000 per seed

Evaluation result

The final evidence compares DQN, heuristic, and random policies on the same held-out simulator seeds. DQN beats random on every metric but does not beat the heuristic baseline.

Metric	DQN mean	Heuristic mean	Random mean	Winner
Reward	0.714965	0.871716	-10.769327	heuristic
Final fatigue	0.535816	0.528991	0.882249	heuristic

Metric	DQN mean	Heuristic mean	Random mean	Winner
Final risk	0.235068	0.234774	0.715206	heuristic
Repeated mistake rate	0.000556	0.000000	0.375000	heuristic
Correction rate	0.440000	0.427778	0.897222	heuristic

Detailed reproducibility evidence is in:

docs/04_rl_training_evidence.md
 outputs/rl_training/

Output artifact

models/coach_policy.zip
 models/rl_runs/coach_policy_seed_*.zip
 outputs/rl_training/episode_rewards.csv
 outputs/rl_training/policy_comparison.csv

If DQN cannot run, fallback metadata:

models/coach_policy.json

Safety policy

DQN output is not blindly used.

Safety override triggers when:

Condition	Override reason
<code>fatigue_score >= 0.65</code>	Rest may be safer
<code>injury_risk >= 0.45</code>	Joint highlight or correction needed
<code>mistake_label</code> is knee tracking, forward lean, unstable motion, or shallow squat	Known form issue requires targeted cue

Why safety layer exists

RL trained on synthetic trajectories can choose suboptimal actions in real videos.
 Safety policy protects demo behavior and makes decisions explainable.

Heuristic fallback policy

Fallback policy maps conditions to actions:

Condition	Action
High fatigue	<code>recommend_rest</code>
High injury risk	<code>joint_highlight</code>
Unstable motion	<code>slow_tempo</code>
Shallow squat	<code>adjust_difficulty</code>
Forward lean or knee tracking	<code>joint_highlight</code>
No issue	<code>no_feedback</code>

Why kept:

- Makes app reliable even if RL package is missing.
- Gives deterministic baseline for tests.
- Helps compare model policy with safety-filtered final action.

Datasets used

1. User-provided real squat video

Path used during QA:

`/Users/tarry/Downloads/e8994a4c-7f2f-424e-b641-751ed98843e2.mov`

Bundled demo path:

`data/sample_videos/real_squat_sample.mov`

Generated outputs:

`outputs/real_video_session/latest_session/landmarks.csv`
`outputs/real_video_session/latest_session/rep_features.csv`
`outputs/real_video_session/latest_session/coach_outputs.json`
`outputs/real_video_session/latest_session/pose_overlay.mp4`

Real video results:

Metric	Value
Detected reps	17
Average fatigue	0.341
Average injury risk	0.659
Top mistake	knee tracking
Joint highlight actions	15
Rest recommendations	2
Overlay codec	<code>h264/avc1/yuv420p</code>

Why used:

- Proves real video pipeline.

- Shows MediaPipe pose extraction, rep segmentation, coaching, overlay, and evidence frame generation work end to end.

2. Synthetic squat landmark dataset

Path:

`data/sample_landmarks/synthetic_squat_landmarks.csv`

Generated by:

```
src/synthetic.py
python -m src.generate_sample_assets
```

Synthetic dataset design:

- 5 squat reps.
- 30 FPS.
- 42 frames per rep.
- Uses sinusoidal squat motion.
- Adds fatigue-related drift.
- Adds slight coordinate jitter.
- Generates same 8 landmarks used by real video pipeline.

Why used:

- Reproducible tests.
- No dependency on external public dataset.
- Allows evaluation when no real video is available.
- Gives stable expected rep count for QA.

Synthetic evaluation result:

Metric	Value
Detected reps	5
Expected reps	5
Rep-count accuracy	1.0
Average fatigue	0.089
Average injury risk	0.205
Mistake types	knee tracking: 1

3. Synthetic RL training trajectories

Generated inside:

```
src/rl_env.py
```

Why used:

- Real multi-session user feedback data is not available.

- RL still needs many state-action-reward transitions.
- Simulated environment gives controlled fatigue/risk dynamics.
- Submission can explain RL design without pretending to have clinical-scale dataset.

Trajectory variables:

- Rep progress.
- Knee signal.
- Hip signal.
- Torso signal.
- Fatigue.
- Injury risk.
- Repeated mistake count.
- Previous action.
- Skill encoding.

4. User profile data

Path:

`data/profiles/default_user.json`

Profile fields:

```
user_id
skill_level
height_cm
injury_notes
baseline_angles
history
```

Why used:

- Supports personalization.
- Stores baseline angles used in mistake thresholds.
- Gives ML/RL state vector user context.

Explainability outputs

Each rep writes:

```
state
action
reward
problem
reason
correction
evidence_frame
```

Technical dashboard additionally shows:

- Triggered threshold.
- State vector.
- Raw model action.
- Final action.
- Decision source: `dqn_policy`, `safety_policy`, or fallback policy.
- Decision reason.
- Reward.
- Evidence frame.

What panel can ask and strong answer

Is this machine learning?

Yes. It uses pretrained deep learning pose estimation, engineered ML features, and RL policy training. It is not pure rule-based logic because perception and action policy involve learned models. It also uses rule-based safety for explainability and reliability.

Is this deep learning?

Yes. MediaPipe Pose is deep learning inference for human keypoint detection. Stable-Baselines3 DQN uses neural network Q-function approximation for discrete action selection.

Is this reinforcement learning?

Yes. Coaching is formulated as MDP. State includes biomechanics and user context. Action is coaching feedback. Reward reflects improvement, fatigue, risk, repeated mistakes, and correction cost. DQN learns policy over simulated squat sessions.

Why use synthetic data?

Real longitudinal user feedback data is unavailable for one-week MVP. Synthetic landmark data supports reproducible tests, while real user-provided video validates video path. Synthetic RL trajectories support policy training without making unsupported clinical claims.

Why not train supervised classifier?

A classifier would label each rep independently. This project needs sequential coaching, where previous feedback and fatigue affect next action. RL better matches that problem.

Why add safety policy if DQN exists?

Because real user safety matters. DQN trained from simulated trajectories should not be trusted alone. Safety policy keeps high fatigue and high risk

decisions conservative and explainable.

Limitations to state clearly

- Squat-only prototype.
- Injury risk is proxy, not diagnosis.
- RL training data is simulated.
- Pose quality depends on camera angle, lighting, occlusion, and landmark visibility.
- No clinical validation.
- No live webcam coaching in current MVP.
- No Grad-CAM because system uses pose landmarks and RL state vectors, not image classifier heatmaps.